

Homework3 RNN 实现古诗生成

学号: SA25006010 姓名: 周子涵

一、任务说明与方法概览

数据集由四个 json 文件组成, 即 poet.song, 40k 到 43k。要求: 以“明月”为总起词, 通过训练使模型能够生成固定格式的古诗。在这里, 使 RNN 生成七言绝句。

数据预处理: 要求生成固定格式的古诗, 如五言绝句、七言绝句等, 基于这个要求, 一般而言需要确保使用的训练数据中都是该格式的古诗;

神经网络构建: 在 Python 中使用 Pytorch 等神经网络框架构建课上讲到的LSTM 等 RNN 类型网络, 也可以使用更复杂的网络, 可以选择使用多个隐藏层和不同的激活函数测试效果;

模型测试与评估: 要求绘制一张 loss 图展示构建的神经网络可收敛。

二、代码实现

```
In [13]: # 导入依赖库
import json
import os
import numpy as np
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
```

```
In [14]: # ===== 1. 超参数设置 =====
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
BATCH_SIZE = 64
EMBEDDING_DIM = 128
HIDDEN_DIM = 256
NUM_LAYERS = 2
EPOCHS = 200
LR = 0.001
MAX_LEN = 28 # 七言绝句4句×7字=28字
START_WORD = "明月" # 作业要求总起词
```

```
In [15]: # ===== 2. 数据预处理 =====
def load_data(file_path):
    """加载古诗json数据, 筛选七言绝句"""
    poems = []
    with open(file_path, 'r', encoding='utf-8') as f:
        data = json.load(f)
        for item in data:
            paragraphs = item['paragraphs']
            # 筛选: 2句、每句14字、无特殊符号
            if len(paragraphs) == 2:
                valid = True
                poem = ""
```

```

        for sent in paragraphs:
            sent = sent.strip()
            if len(sent) != 16:
                valid = False
                break
            sent = sent[:7] + sent[8:15]
            poem += sent
            poems.append(poem)
    return poems

def build_vocab(poems):
    """构建汉字-索引映射词典"""
    vocab = set()
    for poem in poems:
        vocab.update(poem)
    vocab = sorted(list(vocab))
    char2idx = {char: i+1 for i, char in enumerate(vocab)} # 0留给填充
    char2idx['<PAD>'] = 0
    idx2char = {i: char for char, i in char2idx.items()}
    return char2idx, idx2char, len(char2idx)

def poem2tensor(poems, char2idx, max_len):
    """古诗转张量"""
    data = []
    for poem in poems:
        idx = [char2idx[c] for c in poem]
        # 填充/截断到固定长度
        if len(idx) < max_len:
            idx += [char2idx['<PAD>']] * (max_len - len(idx))
        else:
            idx = idx[:max_len]
        data.append(idx)
    data = np.array(data, dtype=np.int64)
    assert data.ndim == 2 and data.shape[1] == max_len, f"data shape error: {data.shape}"
    # 输入x=前n-1字, 目标y=后n-1字
    x = torch.tensor(data[:, :-1], dtype=torch.long).to(DEVICE)
    y = torch.tensor(data[:, 1:], dtype=torch.long).to(DEVICE)
    return x, y

# 加载数据集

poem0 = load_data("poet.song.40000.json")
poem1 = load_data("poet.song.41000.json")
poem2 = load_data("poet.song.42000.json")
poem3 = load_data("poet.song.43000.json")
poems = poem0 + poem1 + poem2 + poem3
print(f"加载到诗歌数量: {len(poems)}")
assert len(poems) > 0, "没有加载到任何诗歌, 请检查数据文件或筛选条件!"
char2idx, idx2char, VOCAB_SIZE = build_vocab(poems)
x_data, y_data = poem2tensor(poems, char2idx, MAX_LEN)

```

加载到诗歌数量: 1213

```

In [16]: # ===== 3. LSTM模型定义 =====
class PoemLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers):
        super(PoemLSTM, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_dim, vocab_size)

```

```

def forward(self, x, hidden=None):
    embed = self.embedding(x)
    out, hidden = self.lstm(embed, hidden)
    out = self.fc(out)
    return out, hidden

# 初始化模型
model = PoemLSTM(VOCAB_SIZE, EMBEDDING_DIM, HIDDEN_DIM, NUM_LAYERS).to(DEVICE)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)

# ===== 4. 模型训练 =====
loss_history = []
print("===== 开始训练 =====")
for epoch in range(EPOCHS):
    model.train()
    total_loss = 0
    # 分批训练
    for i in range(0, len(x_data), BATCH_SIZE):
        x_batch = x_data[i:i+BATCH_SIZE]
        y_batch = y_data[i:i+BATCH_SIZE]

        optimizer.zero_grad()
        output, _ = model(x_batch)
        # 调整维度计算损失
        loss = criterion(output.reshape(-1, VOCAB_SIZE), y_batch.reshape(-1))
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    avg_loss = total_loss / (len(x_data) // BATCH_SIZE + 1)
    loss_history.append(avg_loss)
    print(f"Epoch [{epoch+1}/{EPOCHS}], Average Loss: {avg_loss:.4f}")

# ===== 5. 绘制Loss曲线 =====
plt.figure(figsize=(10, 5))
plt.plot(range(1, EPOCHS+1), loss_history, 'b-', linewidth=2)
plt.title('Training Loss Curve')
plt.xlabel('Epoch')
plt.ylabel('Train Loss')
plt.grid(True)
plt.savefig('loss_curve.png') # 保存图片
plt.show()

```

===== 开始训练 =====

Epoch [1/200], Average Loss: 6.2360
Epoch [2/200], Average Loss: 5.3396
Epoch [3/200], Average Loss: 5.2402
Epoch [4/200], Average Loss: 5.2194
Epoch [5/200], Average Loss: 5.2045
Epoch [6/200], Average Loss: 5.1941
Epoch [7/200], Average Loss: 5.1825
Epoch [8/200], Average Loss: 5.1672
Epoch [9/200], Average Loss: 5.1458
Epoch [10/200], Average Loss: 5.1189
Epoch [11/200], Average Loss: 5.0982
Epoch [12/200], Average Loss: 5.0679
Epoch [13/200], Average Loss: 5.0278
Epoch [14/200], Average Loss: 4.9889
Epoch [15/200], Average Loss: 4.9366
Epoch [16/200], Average Loss: 4.8830
Epoch [17/200], Average Loss: 4.8176
Epoch [18/200], Average Loss: 4.7273
Epoch [19/200], Average Loss: 4.6347
Epoch [20/200], Average Loss: 4.5383
Epoch [21/200], Average Loss: 4.4416
Epoch [22/200], Average Loss: 4.3429
Epoch [23/200], Average Loss: 4.2319
Epoch [24/200], Average Loss: 4.1204
Epoch [25/200], Average Loss: 4.0270
Epoch [26/200], Average Loss: 3.9101
Epoch [27/200], Average Loss: 3.7802
Epoch [28/200], Average Loss: 3.6590
Epoch [29/200], Average Loss: 3.5466
Epoch [30/200], Average Loss: 3.4161
Epoch [31/200], Average Loss: 3.2834
Epoch [32/200], Average Loss: 3.1469
Epoch [33/200], Average Loss: 3.0199
Epoch [34/200], Average Loss: 2.8917
Epoch [35/200], Average Loss: 2.7384
Epoch [36/200], Average Loss: 2.6116
Epoch [37/200], Average Loss: 2.4642
Epoch [38/200], Average Loss: 2.3316
Epoch [39/200], Average Loss: 2.1959
Epoch [40/200], Average Loss: 2.0782
Epoch [41/200], Average Loss: 1.9573
Epoch [42/200], Average Loss: 1.8562
Epoch [43/200], Average Loss: 1.7464
Epoch [44/200], Average Loss: 1.6423
Epoch [45/200], Average Loss: 1.5356
Epoch [46/200], Average Loss: 1.4379
Epoch [47/200], Average Loss: 1.3366
Epoch [48/200], Average Loss: 1.2641
Epoch [49/200], Average Loss: 1.1961
Epoch [50/200], Average Loss: 1.1352
Epoch [51/200], Average Loss: 1.0603
Epoch [52/200], Average Loss: 0.9811
Epoch [53/200], Average Loss: 0.9051
Epoch [54/200], Average Loss: 0.8268
Epoch [55/200], Average Loss: 0.7547
Epoch [56/200], Average Loss: 0.6881
Epoch [57/200], Average Loss: 0.6409
Epoch [58/200], Average Loss: 0.5932
Epoch [59/200], Average Loss: 0.5628

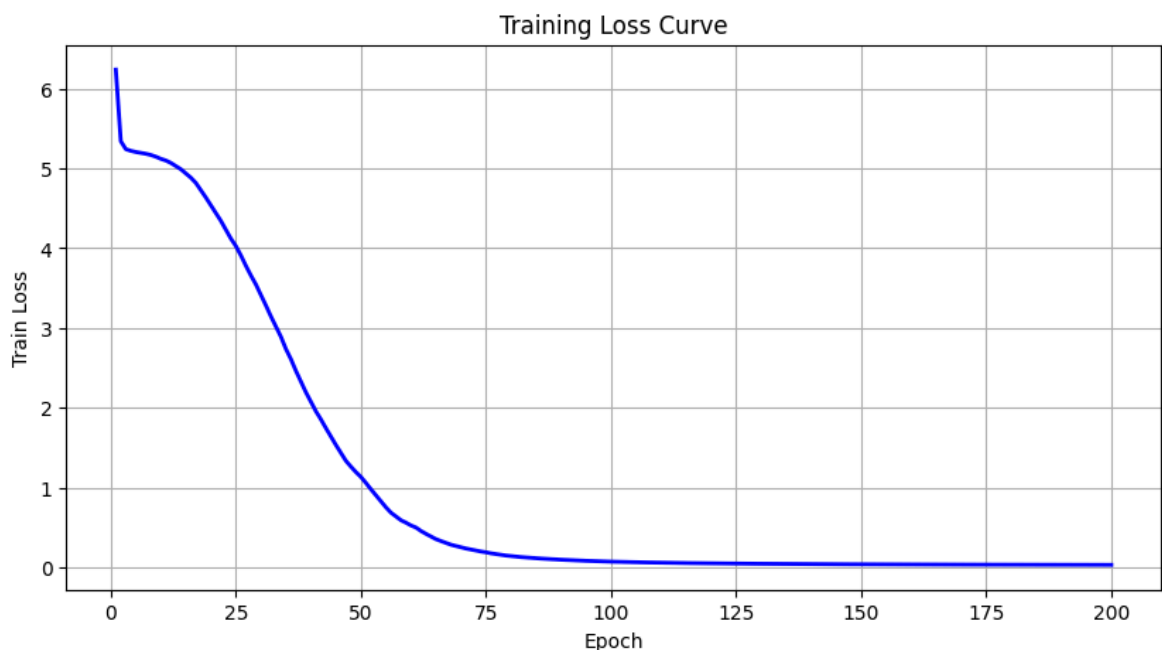
Epoch [60/200], Average Loss: 0.5283
Epoch [61/200], Average Loss: 0.5018
Epoch [62/200], Average Loss: 0.4586
Epoch [63/200], Average Loss: 0.4227
Epoch [64/200], Average Loss: 0.3895
Epoch [65/200], Average Loss: 0.3560
Epoch [66/200], Average Loss: 0.3319
Epoch [67/200], Average Loss: 0.3092
Epoch [68/200], Average Loss: 0.2864
Epoch [69/200], Average Loss: 0.2705
Epoch [70/200], Average Loss: 0.2540
Epoch [71/200], Average Loss: 0.2391
Epoch [72/200], Average Loss: 0.2265
Epoch [73/200], Average Loss: 0.2147
Epoch [74/200], Average Loss: 0.2015
Epoch [75/200], Average Loss: 0.1918
Epoch [76/200], Average Loss: 0.1790
Epoch [77/200], Average Loss: 0.1691
Epoch [78/200], Average Loss: 0.1595
Epoch [79/200], Average Loss: 0.1512
Epoch [80/200], Average Loss: 0.1441
Epoch [81/200], Average Loss: 0.1377
Epoch [82/200], Average Loss: 0.1318
Epoch [83/200], Average Loss: 0.1268
Epoch [84/200], Average Loss: 0.1218
Epoch [85/200], Average Loss: 0.1174
Epoch [86/200], Average Loss: 0.1133
Epoch [87/200], Average Loss: 0.1094
Epoch [88/200], Average Loss: 0.1057
Epoch [89/200], Average Loss: 0.1023
Epoch [90/200], Average Loss: 0.0990
Epoch [91/200], Average Loss: 0.0959
Epoch [92/200], Average Loss: 0.0929
Epoch [93/200], Average Loss: 0.0901
Epoch [94/200], Average Loss: 0.0875
Epoch [95/200], Average Loss: 0.0850
Epoch [96/200], Average Loss: 0.0827
Epoch [97/200], Average Loss: 0.0805
Epoch [98/200], Average Loss: 0.0784
Epoch [99/200], Average Loss: 0.0765
Epoch [100/200], Average Loss: 0.0747
Epoch [101/200], Average Loss: 0.0730
Epoch [102/200], Average Loss: 0.0714
Epoch [103/200], Average Loss: 0.0699
Epoch [104/200], Average Loss: 0.0685
Epoch [105/200], Average Loss: 0.0672
Epoch [106/200], Average Loss: 0.0659
Epoch [107/200], Average Loss: 0.0647
Epoch [108/200], Average Loss: 0.0636
Epoch [109/200], Average Loss: 0.0625
Epoch [110/200], Average Loss: 0.0615
Epoch [111/200], Average Loss: 0.0605
Epoch [112/200], Average Loss: 0.0596
Epoch [113/200], Average Loss: 0.0587
Epoch [114/200], Average Loss: 0.0578
Epoch [115/200], Average Loss: 0.0570
Epoch [116/200], Average Loss: 0.0562
Epoch [117/200], Average Loss: 0.0555
Epoch [118/200], Average Loss: 0.0547
Epoch [119/200], Average Loss: 0.0541

Epoch [120/200], Average Loss: 0.0534
Epoch [121/200], Average Loss: 0.0528
Epoch [122/200], Average Loss: 0.0522
Epoch [123/200], Average Loss: 0.0516
Epoch [124/200], Average Loss: 0.0510
Epoch [125/200], Average Loss: 0.0505
Epoch [126/200], Average Loss: 0.0499
Epoch [127/200], Average Loss: 0.0494
Epoch [128/200], Average Loss: 0.0489
Epoch [129/200], Average Loss: 0.0485
Epoch [130/200], Average Loss: 0.0480
Epoch [131/200], Average Loss: 0.0476
Epoch [132/200], Average Loss: 0.0472
Epoch [133/200], Average Loss: 0.0468
Epoch [134/200], Average Loss: 0.0464
Epoch [135/200], Average Loss: 0.0460
Epoch [136/200], Average Loss: 0.0456
Epoch [137/200], Average Loss: 0.0453
Epoch [138/200], Average Loss: 0.0449
Epoch [139/200], Average Loss: 0.0446
Epoch [140/200], Average Loss: 0.0443
Epoch [141/200], Average Loss: 0.0440
Epoch [142/200], Average Loss: 0.0436
Epoch [143/200], Average Loss: 0.0433
Epoch [144/200], Average Loss: 0.0430
Epoch [145/200], Average Loss: 0.0428
Epoch [146/200], Average Loss: 0.0425
Epoch [147/200], Average Loss: 0.0422
Epoch [148/200], Average Loss: 0.0419
Epoch [149/200], Average Loss: 0.0417
Epoch [150/200], Average Loss: 0.0415
Epoch [151/200], Average Loss: 0.0412
Epoch [152/200], Average Loss: 0.0410
Epoch [153/200], Average Loss: 0.0408
Epoch [154/200], Average Loss: 0.0405
Epoch [155/200], Average Loss: 0.0403
Epoch [156/200], Average Loss: 0.0401
Epoch [157/200], Average Loss: 0.0399
Epoch [158/200], Average Loss: 0.0397
Epoch [159/200], Average Loss: 0.0395
Epoch [160/200], Average Loss: 0.0393
Epoch [161/200], Average Loss: 0.0392
Epoch [162/200], Average Loss: 0.0390
Epoch [163/200], Average Loss: 0.0388
Epoch [164/200], Average Loss: 0.0386
Epoch [165/200], Average Loss: 0.0385
Epoch [166/200], Average Loss: 0.0383
Epoch [167/200], Average Loss: 0.0381
Epoch [168/200], Average Loss: 0.0380
Epoch [169/200], Average Loss: 0.0378
Epoch [170/200], Average Loss: 0.0377
Epoch [171/200], Average Loss: 0.0375
Epoch [172/200], Average Loss: 0.0374
Epoch [173/200], Average Loss: 0.0373
Epoch [174/200], Average Loss: 0.0371
Epoch [175/200], Average Loss: 0.0370
Epoch [176/200], Average Loss: 0.0368
Epoch [177/200], Average Loss: 0.0367
Epoch [178/200], Average Loss: 0.0366
Epoch [179/200], Average Loss: 0.0365

```

Epoch [180/200], Average Loss: 0.0364
Epoch [181/200], Average Loss: 0.0362
Epoch [182/200], Average Loss: 0.0361
Epoch [183/200], Average Loss: 0.0360
Epoch [184/200], Average Loss: 0.0359
Epoch [185/200], Average Loss: 0.0358
Epoch [186/200], Average Loss: 0.0357
Epoch [187/200], Average Loss: 0.0356
Epoch [188/200], Average Loss: 0.0355
Epoch [189/200], Average Loss: 0.0354
Epoch [190/200], Average Loss: 0.0353
Epoch [191/200], Average Loss: 0.0352
Epoch [192/200], Average Loss: 0.0351
Epoch [193/200], Average Loss: 0.0350
Epoch [194/200], Average Loss: 0.0349
Epoch [195/200], Average Loss: 0.0348
Epoch [196/200], Average Loss: 0.0347
Epoch [197/200], Average Loss: 0.0347
Epoch [198/200], Average Loss: 0.0346
Epoch [199/200], Average Loss: 0.0345
Epoch [200/200], Average Loss: 0.0344

```



```

In [17]: # ===== 6. 古诗生成（明月开头·七言绝句） =====
def generate_poem(start_word, model, char2idx, idx2char, max_len=28):
    model.eval()
    poem = list(start_word)
    hidden = None
    with torch.no_grad():
        for _ in range(max_len - len(start_word)):
            x = torch.tensor([[char2idx[c] for c in poem]], dtype=torch.long).to(
                output, hidden = model(x, hidden)
            )
            # 取最后一个字的预测概率
            prob = output.argmax(-1)[0].item()
            poem.append(idx2char[prob])
    # 按7字一句拆分
    poem = ''.join(poem)
    return [poem[i*7:(i+1)*7] for i in range(4)]

# 生成古诗
print("\n===== 生成古诗 =====")

```

```
result = generate_poem(START_WORD, model, char2idx, idx2char)
for idx, sent in enumerate(result, 1):
    print(f"{sent}")
```

===== 生成古诗 =====

明月圓如何事更
滿庭春歸回雪頭
雲盡未未知思留
鳴今夜堂有靜只